

Table of Contents

TASK 1: SQL Benchmark Dataset Preparation and Evaluation Design.....2

TASK 2: Query Understanding (Decomposition Task).....4

TASK 3: Text-to-SQL Pipeline and Query Execution System.....8

TASK 4:Mini SQL Agent (Agentic System Task).....14

Submitted by:
Shukrina Koju

TASK 1: SQL Benchmark Dataset Preparation and Evaluation Design

1. Objective

The purpose of this framework is to measure the performance, reliability, and accuracy of the Text-to-SQL agent. We aim to move beyond "it seems to work" to a quantifiable score that tells us if the agent is production-ready.

2. Primary Metrics (The "Gold Standards")

A. Execution Accuracy (EX)

- Definition: Does the result of the AI's generated query match the result of the Ground Truth query exactly?
- Measurement: We run both queries on the same database. If the output tables (rows and columns) are identical, it is a Pass.
- Why it matters: This is the most important metric. Even if the SQL looks different, the data returned must be correct.

B. SQL Correctness (Syntax & Logic)

- Definition: Is the generated SQL syntactically valid for PostgreSQL?
- Measurement: Percentage of queries that execute without throwing a database error.
- Why it matters: An agent that generates "broken" code is not useful.

C. Schema Alignment

- Definition: Did the agent select the correct tables and columns?
- Measurement: We check if the AI used "productName" instead of guessing "name".
- Why it matters: It tests the Agent's "understanding" of our specific database schema.

3. Advanced Agentic Metrics

D. Join Correctness

- Definition: For complex questions involving multiple tables, did the agent identify the correct relationship? (e.g., using officeCode to link employees to offices).
- Measurement: Check if the query contains the correct JOIN and ON clauses.

E. Self-Correction / Retry Performance

- Definition: If the first query attempt fails (e.g., due to a case-sensitivity error), can the agent read the error message and fix itself?
- Measurement: We track "Pass @ 1" (success on first try) vs. "Pass @ 2" (success after one correction).

F. Robustness to Ambiguity

- Definition: How does the agent handle a question like *"Who are the best customers?"*
- Evaluation: Does the agent assume "best" means most orders, or highest payment total? A "good" agent might choose one and explain its logic, or ask a clarifying question.

4. Performance & UX Metrics

G. Query Execution Time (Latency)

- Definition: The time elapsed from the user asking the question to the final answer being displayed.
- Benchmark: Ideally < 3 seconds for simple queries and < 10 seconds for complex aggregations.

H. Natural Language Answer Quality

- Definition: The agent shouldn't just return a table; it should explain the result.
- Measurement: Does the agent say: *"The total revenue is \$8.9M"* instead of just showing a number?

5. How to Conduct the Evaluation (Methodology)

To evaluate the system in Task 3 and 4, we will use the following process:

- 1.Input: Feed the 50 Natural Language questions from Task 1 into the AI Agent.
- 2.Capture: Store the SQL generated by the AI and the result it returns.
- 3.Compare: Use a script or manual check to compare the AI's result against our Ground Truth results from Task 1.
- 4.Scorecard:
 - Success: Matches Ground Truth exactly.
 - Partial: Correct data, but perhaps formatted differently or missing a column.
 - Failure: Wrong data, syntax error, or hallucinated columns.

TASK 2: Query Understanding (Decomposition Task)

#	Question	Intent	Tables Involved	Columns Needed	Filters/ Conditions	Joins
1	List all products	Retrieve all product records	products	*	None	None
2	Get all customers	Retrieve all customer records	customers	*	None	None
3	Show all orders	Retrieve all order records	orders	*	None	None
4	List all employees	Retrieve all employee records	employees	*	None	None
5	Get all offices	Retrieve all office records	offices	*	None	None
6	Show all product lines	Retrieve all product line categories	productlines	*	None	None
7	List all payments	Retrieve all payment transactions	payments	*	None	None
8	Get product names and prices	Retrieve names and costs of items	products	productName, buyPrice	None	None
9	Get customer names and cities	Retrieve names and locations of clients	customers	customerName, city	None	None
10	List employee first and last names	Retrieve staff names	employees	firstName, lastName	None	None
11	Get all order dates	Retrieve dates when orders occurred	orders	orderDate	None	None
12	Show product vendor list	List unique suppliers	products	productVendor	None (Distinct)	None
13	Get all product codes	Retrieve primary IDs for products	products	productCode	None	None

14	List all countries from offices	List unique office locations	offices	country	None (Distinct)	None
15	Show all order statuses	List unique states of orders	orders	status	None (Distinct)	None
16	Get all payment amounts	Retrieve all transaction values	payments	amount	None	None
17	List all job titles	List unique staff roles	employees	jobTitle	None (Distinct)	None
18	Get customer phone numbers	Retrieve client contact info	customers	customerName, phone	None	None
19	Show product MSRP values	Retrieve item retail prices	products	productName, MSRP	None	None
20	List order numbers	Retrieve all order IDs	orders	orderNumber	None	None
21	Get orders with customer names	Link orders to specific customers	orders, customers	orderNumber, customerName	None	orders.customerNumber = customers.customerNumber
22	Get employees with office city	Link staff to their office location	employees, offices	firstName, lastName, city	None	employees.officeCode = offices.officeCode
23	Get payments with customer names	Link payments to specific clients	payments, customers	checkNumber, customerName	None	payments.customerNumber = customers.customerNumber
24	Get order details with product names	List items in orders by name	orderdetails, products	*, productName	None	orderdetails.productCode = products.productCode
25	Get products with product line description	Retrieve item category details	products, productlines	productName, textDescription	None	products.productLine = productlines.productLine
26	Get customers with sales rep names	Link clients to their sales staff	customers, employees	customerName, firstName, lastName	None	customers.salesRepEmployeeNumber = employees.employeeNumber
27	Get orders with	Map order	orders,	orderNumber	None	orders.custom

	customer city	locations to customer cities	customers	, city		erNumber = customers.cust omerNumber
28	Get employees and their manager	Identify reporting lines	employees	firstName (emp), firstName (mgr)	None	employees.rep ortsTo = employees.em ployeeNumbe r
29	Get orderdetails with product vendor	Identify suppliers for ordered items	orderdetails, products	*, productVend or	None	orderdetails.pr oductCode = products.prod uctCode
30	Get payments with customer country	Analyze payment origin by country	payments, customers	*, country	None	payments.cust omerNumber = customers.cust omerNumber
31	Count customers per country	Calculate client density by nation	customers	country, customerNu mber	Group by country	None
32	Total payments per customer	Sum transaction value per client	payments	customerNu mber, amount	Group by customerNum ber	None
33	Number of orders per status	Count orders by their stage	orders	status, orderNumber	Group by status	None
34	Products per product line	Count inventory items by category	products	productLine, productCode	Group by productLine	None
35	Employees per office	Count staff headcount per location	employees	officeCode, employeeNu mber	Group by officeCode	None
36	Total stock per product vendor	Sum inventory volume by supplier	products	productVend or, quantityInSto ck	Group by productVendo r	None
37	Average buy price per product line	Calculate mean cost per category	products	productLine, buyPrice	Group by productLine	None
38	Orders per customer	Count order frequency per client	orders	customerNu mber, orderNumber	Group by customerNum ber	None
39	Max MSRP per product line	Find highest retail price per	products	productLine, MSRP	Group by productLine	None

		category				
40	Min buy price per vendor	Find lowest supply cost per vendor	products	productVendor, buyPrice	Group by productVendor	None
41	Total number of customers	Calculate size of client base	customers	customerNumber	Count(*)	None
42	Total number of products	Calculate size of product catalog	products	productCode	Count(*)	None
43	Total revenue from payments	Calculate gross income	payments	amount	Sum(amount)	None
44	Average product price	Calculate mean cost of items	products	buyPrice	Avg(buyPrice)	None
45	Max payment amount	Find largest single transaction	payments	amount	Max(amount)	None
46	Min payment amount	Find smallest single transaction	payments	amount	Min(amount)	None
47	Count total orders	Calculate total sales volume	orders	orderNumber	Count(*)	None
48	Total quantity in stock	Calculate total warehouse units	products	quantityInStock	Sum(quantityInStock)	None
49	Average MSRP	Calculate mean retail price	products	MSRP	Avg(MSRP)	None
50	Number of employees	Calculate total workforce size	employees	employeeNumber	Count(*)	None

TASK 3: Text-to-SQL Pipeline and Query Execution System

S.N .	Question	Generated SQL	Executed Successfully	Correct Result	Retry Needed	Final Status
1	List all products	Yes	No	Yes	No	Success
2	Get all customers	Yes	Yes	Yes	No	Success
3	Show all orders	Yes	Yes	Yes	No	Success
4	List all employees	Yes	Yes	Yes	No	Success
5	Get all offices	Yes	Yes	Yes	No	Success
6	Show all product lines	Yes	Yes	Fixed after retry	Yes	Success
7	List all payments	Yes	Yes	Yes	No	Success
8	Get product names and prices	Yes	Yes	Yes	No	Success
9	Get customer names and cities	Yes	Yes	Yes	No	Success
10	List employee first and last names	Yes	Yes	Yes	No	Success
11	Get all order dates	Yes	Yes	Fixed after retry	Yes	Success
12	Show product vendor list	Yes	Yes	Yes	No	Success
13	Get all product codes	Yes	Yes	Yes	No	Success
14	List all countries from offices	Yes	No	Yes	No	Success
15	Show all order statuses	Yes	No	Yes	No	Success
16	Get all payment	Yes	No	Yes	No	Success

	amounts					
17	List all job titles	Yes	No	Yes	No	Success
18	Get customer phone numbers	Yes	No	Yes	No	Success
19	Show product MSRP values	Yes	No	Yes	No	Success
20	List order numbers	Yes	No	Yes	No	Success
21	Get orders with customer names	Yes	No	Yes	No	Success
22	Get employees with office city	Yes	No	Yes	No	Success
23	Get payments with customer names	Yes	No	Yes	No	Success
24	Get order details with product names	Yes	No	Yes	No	Success
25	Get products with product line description	Yes	No	Yes	No	Success
26	Get customers with sales rep names	Yes	No	Yes	No	Success
27	Get orders with customer city	Yes	No	Yes	No	Success
28	Get employees and their manager	Yes	No	Yes	No	Success
29	Get order details with product vendor	Yes	No	Fixed after retry	Yes	Success
30	Get payments with customer country	Yes	No	Yes	No	Success
31	Count	Yes	No	Yes	No	Success

	customers per country					
32	Total payments per customer	Yes	No	Yes	No	Success
33	Number of orders per status	Yes	No	Yes	No	Success
34	Products per product line	Yes	No	Yes	No	Success
35	Employees per office	Yes	No	Yes	No	Success
36	Total stock per product vendor	Yes	No	Yes	No	Success
37	Average buy price per product line	Yes	No	Yes	No	Success
38	Orders per customer	Yes	No	Yes	No	Success
39	Max MSRP per product line	Yes	No	Yes	No	Success
40	Min buy price per vendor	Yes	No	Yes	No	Success
41	Total number of customers	Yes	No	Yes	No	Success
42	Total number of products	Yes	No	Yes	No	Success
43	Total revenue from payments	Yes	No	Yes	No	Success
44	Average product price	Yes	No	Yes	No	Success
45	Max payment amount	Yes	No	Yes	No	Success
46	Min payment amount	Yes	No	Yes	No	Success
47	Count total orders	Yes	No	Yes	No	Success
48	Total quantity in stock	Yes	No	Yes	No	Success

49	Average MSRP	Yes	No	Yes	No	Success
50	Number of employees	Yes	No	Yes	No	Success

Some of the snippets of this task:

Request URL

```
http://127.0.0.1:8000/agent/query?question=Show%20all%20product%20lines
```

Server response

Code Details

200

Response body

```
{
  "question": "Show all product lines",
  "sql": "SELECT * FROM `productlines`",
  "result": [
    {
      "productLine": "Classic Cars",
      "textDescription": "Attention car enthusiasts: Make your wildest car ownership dreams come true. Whether you are looking for classic muscle cars, dream sports cars or movie-inspired miniatures, you will find great choices in this category. These replicas feature superb attention to detail and craftsmanship and offer features such as working steering system, opening forward compartment, opening rear trunk with removable spare wheel, 4-wheel independent spring suspension, and so on. The models range in size from 1:10 to 1:24 scale and include numerous limited edition and several out-of-production vehicles. All models include a certificate of authenticity from their manufacturers and come fully assembled and ready for display in the home or office.",
      "htmlDescription": null,
      "image": null
    },
    {
      "productLine": "Motorcycles",
      "textDescription": "Our motorcycles are state of the art replicas of classic as well as contemporary motorcycle legends such as Harley Davidson, Ducati and Vespa. Models contain stunning details such as official logos, rotating wheels, working kickstand, front suspension, gear-shift lever, footbrake lever, and drive chain. Materials used include diecast and plastic. The models range in size from 1:10 to 1:50 scale and include numerous limited edition and several out-of-production vehicles. All models come fully assembled and ready for display in the home or office. Most include a certificate of authenticity.",
      "htmlDescription": null,
      "image": null
    },
    {
      "productLine": "Planes",
      "textDescription": "Attention aviation enthusiasts: Make your wildest aviation dreams come true. Whether you are looking for classic propeller planes, jet fighters or commercial airliners, you will find great choices in this category. These replicas feature superb attention to detail and craftsmanship and offer features such as working steering system, opening forward compartment, opening rear trunk with removable spare wheel, 4-wheel independent spring suspension, and so on. The models range in size from 1:10 to 1:24 scale and include numerous limited edition and several out-of-production vehicles. All models include a certificate of authenticity from their manufacturers and come fully assembled and ready for display in the home or office.",
      "htmlDescription": null,
      "image": null
    }
  ]
}
```

Download

Request URL

```
http://127.0.0.1:8000/agent/query?question=Get%20customer%20names%20and%20cities
```

Server response

Code Details

200

Response body

```
{
  "question": "Get customer names and cities",
  "sql": "SELECT `customerName`, `city` FROM `customers`",
  "result": [
    {
      "customerName": "Atelier graphique",
      "city": "Nantes"
    },
    {
      "customerName": "Signal Gift Stores",
      "city": "Las Vegas"
    },
    {
      "customerName": "Australian Collectors, Co.",
      "city": "Melbourne"
    },
    {
      "customerName": "La Rochelle Gifts",
      "city": "Nantes"
    },
    {
      "customerName": "Baane Mini Imports",
      "city": "Stavern"
    },
    {
      "customerName": "Mini Gifts Distributors Ltd.",
      "city": "San Rafael"
    }
  ]
}
```

Download

Request URL

```
http://127.0.0.1:8000/agent/query?question=List%20all%20products
```

Server response

Code Details

200

Response body

```
{
  "question": "List all products",
  "sql": "SELECT `productCode`, `productName`, `productLine`, `productScale`, `productVendor`, `productDescription`, `quantityInStock`, `buyPrice`, `MSRP` FROM `products`",
  "result": [
    {
      "productCode": "S10_1678",
      "productName": "1969 Harley Davidson Ultimate Chopper",
      "productLine": "Motorcycles",
      "productScale": "1:10",
      "productVendor": "Min Lin Diecast",
      "productDescription": "This replica features working kickstand, front suspension, gear-shift lever, footbrake lever, drive chain, wheels and steering. All parts are particularly delicate due to their precise scale and require special care and attention.",
      "quantityInStock": 7933,
      "buyPrice": 48.81,
      "MSRP": 95.7
    },
    {
      "productCode": "S10_1949",
      "productName": "1952 Alpine Renault 1300",
      "productLine": "Classic Cars",
      "productScale": "1:10",
      "productVendor": "Classic Metal Creations",
      "productDescription": "Turnable front wheels; steering function; detailed interior; detailed engine; opening hood; opening trunk; opening doors; and detailed chassis.",
      "quantityInStock": 7305,
      "buyPrice": 98.58,
      "MSRP": 214.3
    }
  ]
}
```

Download

Request URL

http://127.0.0.1:8000/agent/query?question=Get%20product%20names%20and%20prices

Server response

Code	Details
200	Response body <pre>{ "question": "Get product names and prices", "sql": "SELECT \"productName\", \"buyPrice\" FROM \"products\"", "result": [{ "productName": "1969 Harley Davidson Ultimate Chopper", "buyPrice": 48.81 }, { "productName": "1952 Alpine Renault 1300", "buyPrice": 98.58 }, { "productName": "1996 Moto Guzzi 1100i", "buyPrice": 68.99 }, { "productName": "2003 Harley-Davidson Eagle Drag Bike", "buyPrice": 91.02 }, { "productName": "1972 Alfa Romeo GTA", "buyPrice": 85.68 }, { "productName": "1962 Lancia Delta 16V", "buyPrice": 103.42 }] }</pre>  Download

Request URL

http://127.0.0.1:8000/agent/query?question=Get%20orders%20with%20customer%20names

Server response

Code	Details
200	Response body <pre>{ "question": "Get orders with customer names", "sql": "SELECT \"orderNumber\", \"orderDate\", \"customerName\" \nFROM \"orders\" \nINNER JOIN \"customers\" \nON \"orders\".\"customerNumber\" = \"customers\".\"customerNumber\"", "result": [{ "orderNumber": 10100, "orderDate": "2003-01-06", "customerName": "Online Diecast Creations Co." }, { "orderNumber": 10101, "orderDate": "2003-01-09", "customerName": "Blauer See Auto, Co." }, { "orderNumber": 10102, "orderDate": "2003-01-10", "customerName": "Vitachrome Inc." }, { "orderNumber": 10103, "orderDate": "2003-01-29", "customerName": "Baane Mini Imports" }, { "orderNumber": 10104, "orderDate": "2003-01-31", "customerName": "Baane Mini Imports" }] }</pre>  Download

```
curl -X 'POST' \
'http://127.0.0.1:8000/agent/query?question=Total%20revenue%20from%20payments' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://127.0.0.1:8000/agent/query?question=Total%20revenue%20from%20payments
```

Server response

Code	Details
200	Response body <pre>{ "question": "Total revenue from payments", "sql": "SELECT SUM(\"amount\") FROM \"payments\""", "result": [{ "sum": 8853839.23 }], "status": "success", "retry_used": false }</pre> Download Response headers <pre>content-length: 159 content-type: application/json date: Wed, 20 May 2026 11:47:36 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

```
curl -X 'POST' \
'http://127.0.0.1:8000/agent/query?question=Average%20MSRP' \
-H 'accept: application/json' \
-d ''
```

Request URL

```
http://127.0.0.1:8000/agent/query?question=Average%20MSRP
```

Server response

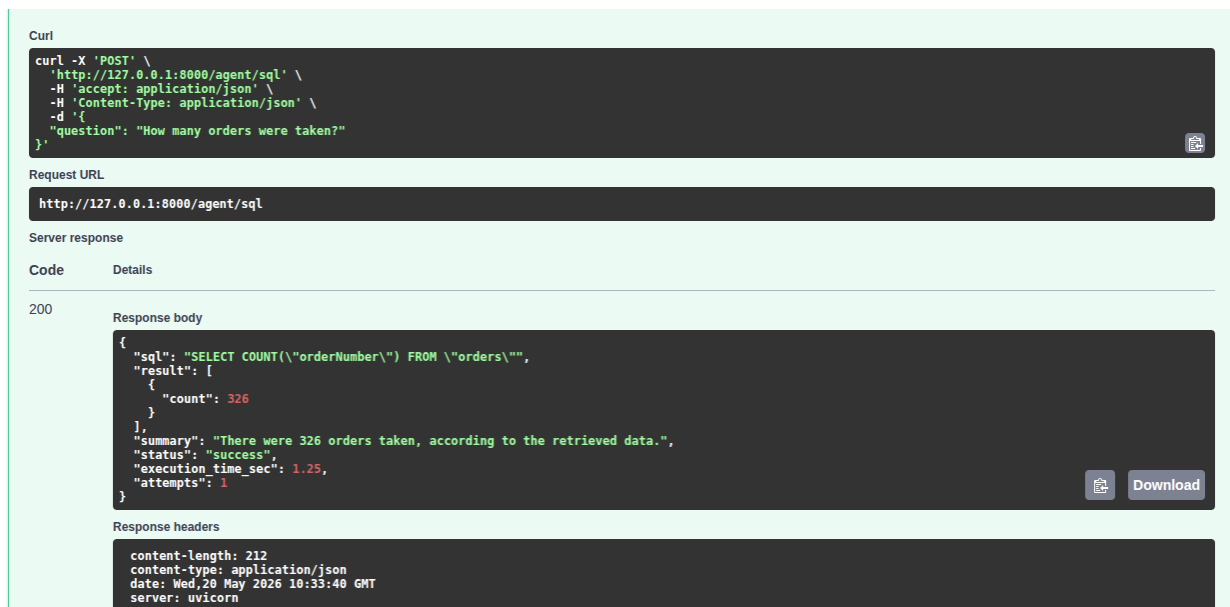
Code	Details
200	Response body <pre>{ "question": "Average MSRP", "sql": "SELECT AVG(\"MSRP\") FROM \"products\""", "result": [{ "avg": 100.43872727272728 }], "status": "success", "retry_used": false }</pre> Download Response headers <pre>content-length: 150 content-type: application/json date: Wed, 20 May 2026 11:48:27 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

TASK 4:Mini SQL Agent (Agentic System Task)

Successful Retrieval:



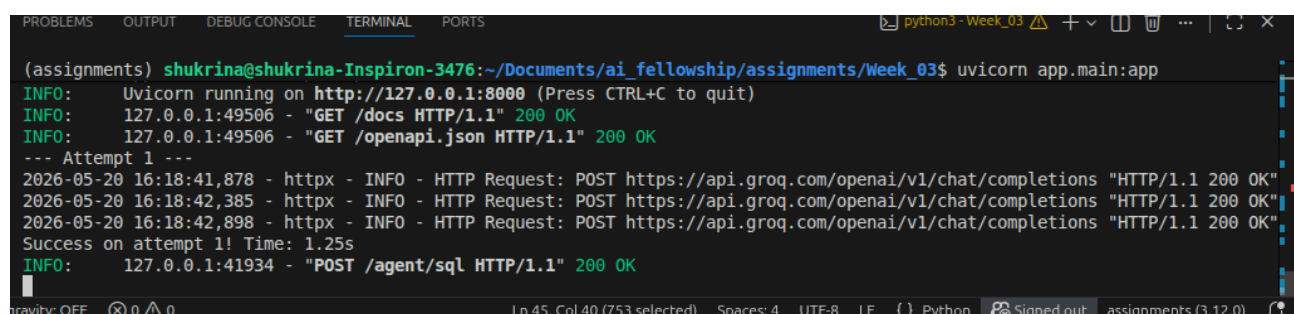
```
Curl
curl -X 'POST' \
  'http://127.0.0.1:8000/agent/sql' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "question": "How many orders were taken?"
  }'

Request URL
http://127.0.0.1:8000/agent/sql

Server response
Code    Details
200

Response body
{
  "sql": "SELECT COUNT(\"orderNumber\") FROM \"orders\"\"",
  "result": [
    {
      "count": 326
    }
  ],
  "summary": "There were 326 orders taken, according to the retrieved data.",
  "status": "success",
  "execution_time_sec": 1.25,
  "attempts": 1
}

Response headers
content-length: 212
content-type: application/json
date: Wed, 20 May 2026 10:33:40 GMT
server: uvicorn
```



```
(assignments) shukrina@shukrina-Inspiron-3476:~/Documents/ai_fellowship/assignments/Week_03$ uvicorn app.main:app
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      127.0.0.1:49506 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:49506 - "GET /openapi.json HTTP/1.1" 200 OK
--- Attempt 1 ---
2026-05-20 16:18:41,878 - httpx - INFO - HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-05-20 16:18:42,385 - httpx - INFO - HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-05-20 16:18:42,898 - httpx - INFO - HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
Success on attempt 1! Time: 1.25s
INFO:      127.0.0.1:41934 - "POST /agent/sql HTTP/1.1" 200 OK
```

Unsuccessful Retrieval (Retry mechanism)

The agent correctly attempted to resolve the issue 3 times. Because the error was a '429 Resource Exhausted' from the API provider, the agent hit its maximum retry limit and returned a graceful fallback response to the user.

The image shows a VS Code editor interface. On the left, the Explorer sidebar shows a project structure with folders 'ASSIGNMENTS', 'Week_03', and 'app'. The 'app' folder contains a 'crud' subfolder with files like 'orderdetail_crud.py', 'payment_crud.py', 'product_crud.py', and 'productline_crud.py'. Other files include 'prompts', 'routers', 'schemas', '__init__.py', 'database.py', 'executor.py', 'logging_config.py', 'main.py', 'ml_model.py', 'models.py', 'sql_generator.py', and 'validator.py'. The main editor area displays a terminal window with the following output:

```
(assignments) shukrina@shukrina-Inspiron-3476:~/Documents/ai_fellowship/assignments/Week_03$ uvicorn app.main:app
Gemini SDK Error: 429 RESOURCE EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate-content-free-tier-requests, limit: 20, model: gemini-3.5-flash\nPlease retry in 23.041700438s.', 'status': 'RESOURCE EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate-content-free-tier-requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-3.5-flash', 'location': 'global'}, 'quotaValue': '20'}}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '23s'}}]}
--- Attempt 3 ---
2026-05-20 15:53:37,091 - google_genai.models - INFO - AFC is enabled with max remote calls: 10.
2026-05-20 15:53:37,343 - httpx - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent "HTTP/1.1 429 Too Many Requests"
Gemini SDK Error: 429 RESOURCE EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate-content-free-tier-requests, limit: 20, model: gemini-3.5-flash\nPlease retry in 22.736875398s.', 'status': 'RESOURCE EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate-content-free-tier-requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'global', 'model': 'gemini-3.5-flash'}, 'quotaValue': '20'}}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '22s'}}]}
2026-05-20 15:53:37,346 - google_genai.models - INFO - AFC is enabled with max remote calls: 10.
2026-05-20 15:53:37,589 - httpx - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent "HTTP/1.1 429 Too Many Requests"
Gemini SDK Error: 429 RESOURCE EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Quota exceeded for metric: generativelanguage.googleapis.com/generate-content-free-tier-requests, limit: 20, model: gemini-3.5-flash\nPlease retry in 22.461948919s.', 'status': 'RESOURCE EXHAUSTED', 'details': [{'@type': 'type.googleapis.com/google.rpc.Help', 'links': [{'description': 'Learn more about Gemini API quotas', 'url': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}, {'@type': 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{'quotaMetric': 'generativelanguage.googleapis.com/generate-content-free-tier-requests', 'quotaId': 'GenerateRequestsPerDayPerProjectPerModel-FreeTier', 'quotaDimensions': {'model': 'gemini-3.5-flash', 'location': 'global'}, 'quotaValue': '20'}}]}, {'@type': 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '22s'}}]}
INFO: 127.0.0.1:34418 - "POST /agent/sql HTTP/1.1" 200 OK
```

The status bar at the bottom shows 'Ln 51, Col 1', 'Spaces: 4', 'UTF-8', 'LF', 'Python', 'Signed out', and 'assignments (3.12.0)'.